CryptOracle: A Modular Framework to Characterize FHE

Cory Brynds
ECE Department
University of Central Florida
Orlando, FL
cory.brynds@ucf.edu

Parker McLeod*
Advanced Micro Devices
Folsom, CA
pmcleod@amd.com

Lauren Caccamise*
ECE Department
Purdue University
West Lafayette, Indiana
lcaccami@purdue.edu

Asmita Pal†
Advanced Micro Devices
Santa Clara, CA
asmitpal@amd.com

Dewan Saiham
CREOL
University of Central Florida
Orlando, FL
dewan.saiham@ucf.edu

Sazadur Rahman
ECE Department
University of Central Florida
Orlando, FL
mohammad.rahman@ucf.edu

Joshua San Miguel
ECE Department
University of Wisconsin-Madison
Madison, WI
jsanmiguel@wisc.edu

Di Wu
ECE Department
University of Central Florida
Orlando, FL
di.wu@ucf.edu

***Abstract*—Privacy-preserving machine learning has become an important long-term pursuit in this era of artificial intelligence (AI). Fully Homomorphic Encryption (FHE) is a uniquely promising solution, offering provable privacy and security guarantees. Unfortunately, computational cost is impeding its mass adoption. Modern solutions are up to six orders of magnitude slower than plaintext execution. Understanding and reducing this overhead is essential to the advancement of FHE, particularly as the underlying algorithms evolve rapidly. This paper presents a detailed characterization of OpenFHE, a comprehensive open-source library for FHE, with a particular focus on the CKKS scheme due to its significant potential for AI and machine learning applications. We introduce CryptOracle, a modular evaluation framework comprising (1) a benchmark suite, (2) a hardware profiler, and (3) a predictive performance model. The benchmark suite encompasses OpenFHE kernels at three abstraction levels: workloads, microbenchmarks, and primitives. The profiler is compatible with standard and user-specified security parameters. CryptOracle monitors application performance, captures microarchitectural events, and logs power and energy usage for AMD and Intel systems. These metrics are consumed by a modeling engine to estimate runtime and energy efficiency across different configuration scenarios, with prediction error ranging from $-7.02\% \sim 8.40\%$ for runtime and $-9.74\% \sim 15.67\%$ for energy (geomean). CryptOracle is open source, fully modular, and serves as a shared platform to facilitate the collaborative advancements of applications, algorithms, software, and hardware in FHE. The CryptOracle code can be accessed at https://github.com/UnaryLab/CryptOracle.**

I. Introduction

Modern computation is increasingly offloaded to the cloud, where shared infrastructure and limited transparency heighten the risk of data compromise. Contemporary security frameworks routinely encrypt data in storage and during transmission; however, they ultimately require plaintext access for computation. This window of exposure allows adversaries to exploit memory access patterns [39], [43], leverage microarchitectural side channels [17], [42], or trigger latent bugs in complex software stacks [11], [19], [35], especially under the proliferation of machine learning based attacks [28], [52].

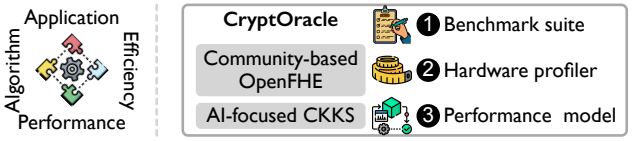

Fig. 1: Overview of this paper.

Fully Homomorphic Encryption (FHE) offers a compelling alternative by enabling computation on encrypted data without decryption [25]. However, this guarantee comes with a much higher cost as compared to other secure-computation paradigms, not just because of the computational intensity but also due to the evolving nature of workloads in the FHE space. Consequently, the systems community has pursued aggressive optimizations to overcome these challenges, catalyzing a vibrant ecosystem of architectural and algorithmic innovations, spanning general-purpose GPU implementations [22] to highly specialized hardware accelerators [2], [33], [34], [49], [50].

Problem Statement. Though existing profilers [24], [30], [44] provide valuable insights, a gap remains in *automating rapid characterization across abstraction levels, hardware vendors, and evaluation metrics*. We highlight challenges as follows.

1) **Long runtime.** FHE operations are intrinsically expensive due to the reliance on modular polynomial arithmetic and number-theoretic transforms, e.g., a multiplication takes 0.156ms in FHE vs. 4.3ns in plaintext [32]. This makes simulation unsuitable, typically adopted in the architectural space, as it would be orders of magnitude slower.

*This work was completed while at the University of Central Florida.
†This work was completed while at the University of Wisconsin-Madison.

2766-0486/26/$31.00 ©2026 IEEE
DOI 10.1109/ISPASS69572.2026.00040

2) **Complicated optimization space.** Performance in FHE is influenced by interdependent factors such as ciphertext size, multiplicative depth, noise growth, parallelism and memory layout. For example, ciphertexts can exceed 50 MB [34] and increase memory pressure; bootstrapping, taking up to 1.5s [49], is needed for noise management. Hence navigating this space involves tuning multiple architectural and algorithmic parameters, making design-space exploration less intuitive.
3) **Rapid algorithmic change.** Due to the strong guarantees of secure computation, the FHE system is evolving rapidly. Frequent updates to encoding strategies and security standards make it difficult to build reusable performance models.

The Solution. We introduce CryptOracle, a modular framework for systematic characterization of FHE applications, as shown in Figure 1. Built atop the community-maintained OpenFHE library [6], CryptOracle lowers the barrier to reproducible evaluation for theorists, system architects, and application developers alike. It comprises:

1) **Benchmark suite.** A top-down collection of representative ML workloads, optimized microbenchmarks, and the semantic primitives (semantically atomic, like multiplication) that underlie them.
2) **Hardware profiler.** An automated tool that extracts runtime, power, and microarchitectural statistics on both AMD and Intel CPUs, averaged across multiple runs under varied parameter sets.
3) **Performance model.** A lightweight additive model that extrapolates primitive-level measurements to predict end-to-end runtime and energy within milliseconds, enabling efficient design-space exploration.

Contributions.

- We present CryptOracle, the first open-source, end-to-end characterization framework for CKKS FHE workloads on commodity CPUs.
- We curate and release a comprehensive benchmark suite spanning popular ML applications, targeted microbenchmarks, and underlying primitives.
- We develop an automated profiler that collects unified performance, power, and security metrics across processor vendors and parameterized inputs.
- We devise a fast prediction model that estimates performance from primitive-level statistics.
- We validate the model experimentally, distill architectural insights, and illustrate how CryptOracle accelerates research on FHE algorithms and systems.

The remainder of the paper is organized as follows. Section II presents the motivation behind this work. Section III introduces the components of the CryptOracle framework, followed by implementation details in Section IV. Section V provides an empirical evaluation of the system. We discuss limitations in Section VI, review prior works in Section VII and conclude in Section VIII.

II. Background and Motivation

A. FHE Framework

Introduced by Gentry in 2009 [25], FHE allows computations to be performed directly on encrypted data, enabling privacy-preserving computation in untrusted environments. Following Gentry's original construction, which was initially kept theoretical due to prohibitive computational overhead, subsequent advancements have led to the development of multiple FHE schemes that improve efficiency and functionality. Notable schemes include BGV [9], BFV [21], FHEW [18], TFHE [15], and CKKS [14].

The increasing interest in deploying FHE in real-world applications has catalyzed the development of several FHE libraries, including Microsoft SEAL [51], PALISADE [46], HEAAN [14], Lattigo [1], HELayers [4] and OpenFHE [6], each supporting a certain number of different schemes. Among these, OpenFHE is one of the most comprehensive libraries. First, OpenFHE supports a wide range of FHE schemes under a unified API. Second, it offers compatibility with the Homomorphic Encryption Standard [36], ensuring cryptographic soundness and reproducibility. Third, OpenFHE allows fine-grained customization of encryption parameters, enabling performance-security tradeoff studies across varied hardware backends. Finally, it features built-in support for both leveled and bootstrapped CKKS, facilitating benchmarking across arbitrary depths. OpenFHE supports a rich set of homomorphic operations, including ciphertext-plaintext and ciphertext-ciphertext addition and multiplication, as well as advanced primitives such as vector rotation, scalar encoding, conjugation, bootstrapping.

B. The CKKS Scheme

The CKKS scheme supports approximate floating-point arithmetic over vectors of complex numbers, making it particularly suited for applications in machine learning, analytics, and scientific computing. Figure 2 illustrates the process. A complex plaintext vector $m \in \mathbb{C}^{N/2}$ is first encoded as the polynomial $m(X) = \sum_{i=0}^{N-1} c_i X^i$ in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N+1)$, where N is a power of two and Q is the ciphertext modulus. Encryption samples a random polynomial $a(X)$ and a small Gaussian error $e(X)$, then outputs the ciphertext $\mathrm{ct} = (b(X), a(X)) = (a(X)\cdot s(X) + m(X) + e(X),\ a(X))$, where $s(X)$ is the secret key. Decryption recovers the message by computing $m'(X) = ct \cdot (1 - s(X)) = m(X) + e(X)$, and $m(X)$ can be obtained by removing the noise term $e(X)$.

Homomorphic operations in CKKS operate over encrypted vectors. With $[\![\mathbf{m}]\!]$ referring to the ciphertext of a plaintext message of $\mathbf{m}$, important operations include:

- **Add**: Given ciphertexts $[\![\mathbf{m}_1]\!]$ and $[\![\mathbf{m}_2]\!]$, produce $[\![\mathbf{m}_1 + \mathbf{m}_2]\!]$, where the addition is performed component-wise over $\mathbb{C}$.
- **Mult**: Given ciphertexts $[\![\mathbf{m}_1]\!]$ and $[\![\mathbf{m}_2]\!]$, produce $[\![\mathbf{m}_1 \circ \mathbf{m}_2]\!]$, where $\circ$ denotes element-wise multiplication.
- **Rotate**: For a ciphertext $[\![\mathbf{m}]\!]$ and an integer k, compute $[\![\phi_k(\mathbf{m})]\!]$, where ϕ_k cyclically rotates

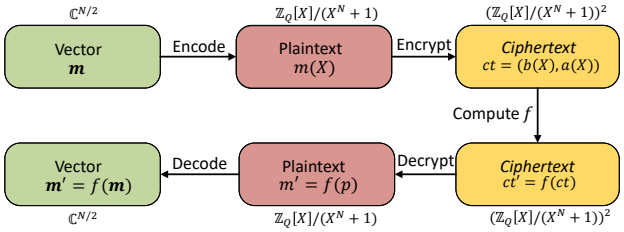

Fig. 2: Overview of CKKS scheme.

the vector by k positions. For example, when $k = 1$: $\mathbf{x} = (x_0, x_1, \ldots, x_{n-2}, x_{n-1})$, and $\phi_1(\mathbf{x}) = (x_{n-1}, x_0, \ldots, x_{n-3}, x_{n-2})$.

CKKS leverages the Number Theoretic Transform (NTT) to accelerate polynomial multiplication: once mapped to the NTT domain, the operation reduces to a point-wise product, reducing the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$. Each homomorphic multiplication amplifies the accumulated noise and simultaneously scales the plaintext. To curb this noise growth, the ciphertext is rescaled by dividing it by its last modulus prime q_L [13]: Rescale(ct) $\rightarrow$ ct$'$ = ct/q_L mod Q', where $Q' = Q/q_L$. After several multiply–rescale rounds, the remaining modulus eventually becomes too small to accommodate further multiplications, capping the achievable circuit depth. Because CKKS does not natively support nonlinear functions (e.g. ReLU), computations rely on polynomial approximations, which typically demand a substantial multiplicative depth [37]. To permit computations of arbitrary depth, CKKS employs *bootstrapping* [25]. This procedure homomorphically evaluates the decryption followed by the re-encryption of a ciphertext, thereby resetting its noise budget and restoring both its modulus and available multiplicative depth. Consequently, encrypted data can undergo an unlimited sequence of operations while still producing correct results. Bootstrapping is indispensable, yet it remains computationally intensive and consumes several levels of the ciphertext modulus. State-of-the-art implementations accelerate bootstrapping through FFTs and polynomial approximations [12], making it the focus of ongoing research in hardware and algorithmic optimizations.

C. Benchmarking and Characterization

Benchmarking and workload characterization are essential for understanding application behavior and for guiding both system- and compiler-level optimizations. Over time, diverse methodologies have been devised to capture the performance characteristics of both general-purpose and domain-specific workloads. General-purpose benchmark suites such as SPEC CPU2017 [10], Graph500 [26], and MLPerf [41] have driven advances in evaluating compute-intensive, irregular-memory-acess, and deep-learning applications. In parallel, specialized characterization frameworks have been proposed to target particular system behaviors. Bircher [8], for instance, leveraged hardware performance counters for comprehensive system-level power estimation, whereas Crape [16] developed a statistically rigorous methodology for profiling dynamic Python workloads. Liu [40] further extended this direction by proposing a data-efficient framework tailored to domain-specific architectures. Despite their breadth, these studies concentrate on conventional, plaintext applications and therefore overlook the unique challenges of FHE, whose encrypted computation, intricate algebraic operations, and unique performance bottlenecks demand specialized profiling strategies.

D. Motivation of CryptOracle

CryptOracle is built on OpenFHE, targeting CKKS across three abstraction levels: workload, microbenchmark, and primitive. The hardware profiler captures CPU behaviors at the primitive level and reports a diverse set of hardware metrics. Using the runtime and energy measured for primitives, the performance model predicts the corresponding values for workloads and microbenchmarks.

Q1: Why benchmark CKKS on OpenFHE?

Recent advances in FHE software and hardware acceleration have moved FHE closer to practical deployment [2], [20], [45], [50]. Even with this progress, the community still lacks a unified, systematic framework for benchmarking.

We adopt OpenFHE because it is community-driven, open source, actively maintained, and executable on widely available CPU platforms. This community-driven backbone of CryptOracle on AMD and Intel processors lets a broad audience study application and algorithm characteristics under identical conditions, promoting fair comparison and accelerating research progress.

Although OpenFHE also implements BGV and BFV, our study focuses on CKKS, the scheme that supports floating-point arithmetic central to AI workloads, to keep workload semantics, runtime behavior, and metric interpretation consistent. Future work may extend the methodology to multiple schemes.

Q2: Why profile FHE at the primitive level?

FHE introduces computational patterns that differ fundamentally from those in conventional benchmarks such as intensive polynomial arithmetic, explicit ciphertext management, and intricate noise control mechanisms.

Where suites such as SPEC or MLPerf evaluate general-purpose programs, FHE workloads intertwine cryptographic kernels with high-dimensional algebraic routines and operations that map poorly to commodity hardware. Consequently, CryptOracle profiles execution at the primitive level. This approach, common in HPC and accelerator design, leverages the consistent and predictable behavior of primitives to guide optimizations across all layers. Profiling with primitive granularity exposes system bottlenecks in fine detail, enabling targeted optimization [22], [31].

Finally, the core CKKS primitives (MULT, RESCALE, ADD, and ROTATE) are expected to remain stable. Future applica-

TABLE I: CryptOracle framework I/O overview.

I/O Type		Variable
Input	Benchmark suite	**FHE abstractions**: FHE workloads, microbenchmarks, primitives **Security parameters**: ciphertext ring dimension, security standard, batch size, computation depth
	Hardware profiler	**FHE setup**: run count for FHE abstractions, a list of abstractions to profile **Thread count**: max allowable OMP threads **Profiling events**: a list of hardware events to record during profiling
	Performance model	**Target abstraction**: microbenchmark or workload **Security parameters**: OpenFHE cryptocontext parameters **Thread count**: number of OMP threads **Operation counts**: FHE operation counts from the workload and microbenchmark **Profiled performance**: runtime, energy of profiled FHE abstractions
Intermediate		Profiling raw recordings
Output		Callstack graph, profiling/prediction results

tions will compose these operations rather than redefine them, so CryptOracle provides enduring value for performance analysis.

Q3: Why model and predict CPU performance?

The high computational cost of FHE has motivated extensive work on custom hardware accelerators [2], [33], [34], [49], [50]. These efforts usually appear alongside new applications and algorithms. Because implementing each idea on actual hardware is complex and time-consuming, assessing the benefits only after deployment is impractical. A faster approach is to estimate performance and efficiency on general-purpose CPUs through analytical or empirical models. Similar modeling has become standard practice for predicting the runtime of AI workloads at various scales [29], [38], [47]. Following this methodology, CryptOracle estimates the performance and energy of FHE applications from primitive-level profiling, enabling accurate yet rapid exploration of algorithmic and application innovations.

III. CryptOracle Framework

A. Overview

Our CryptOracle framework characterizes the performance of FHE through a comprehensive benchmark suite and an integrated hardware profiler. In addition, its performance model enables CryptOracle to rapidly evaluate the performance of the application as the security and hardware parameters are varied. Our CryptOracle framework is shown in Figure 3, with inputs and outputs listed in Table I.

B. Benchmark Suite

The CryptOracle benchmark suite is the primary interface for end users of the framework. It packages a predefined set of FHE abstractions: primitives, microbenchmarks, and workloads, each ready for profiling. After the user supplies the desired security parameters, the suite instructs the orchestrator to perform a combinatorial sweep over the parameters of all abstractions. When multiple abstraction layers are selected, the orchestrator evaluates them in ascending order of complexity: primitives first, followed by microbenchmarks, and finally workloads. For every parameter combination, the orchestrator passes the relevant configuration to the hardware profiler. Table II summarizes all FHE abstractions currently supported.

TABLE II: Supported microbenchmarks and workloads in the CryptOracle benchmark suite, with the default application parameters used for evaluation.

Category	Application	Sec. Std.	N	B	L
Primitive	OpenFHE Primitives	none	2^{16}	2^{12}	10
Micro-benchmark	Matrix Multiplication	none	2^{16}	2^{12}	10
	Logistic Function	none	2^{16}	2^{12}	10
	Sign Evaluation	none	2^{16}	2^{12}	10
Workload	CIFAR-10 Inference	none	2^{16}	2^{12}	5
	Chi Square Test	none	2^{17}	1	3
	ResNet20 Inference	none	2^{16}	2^{14}	10
	Logreg Training	128-bit	2^{17}	2^{16}	14

C. Hardware Profiler

The hardware profiler executes four sequential phases. First, during runtime analysis it measures the application's overall execution time and energy consumption. Second, during event profiling it counts hardware performance events using the same whole-application and setup executions. Third, in the denoising phase it subtracts the setup measurements from both the runtime and event data to obtain metrics that exclude one-time I/O activity and profiling overhead. Finally, it records the application's call stack to support call stack graph generation.

1) Runtime Analysis: During the runtime analysis phase, the hardware profiler records both execution time and energy consumption of the target application with Perf, a user-space Linux utility that interfaces with CPU performance monitoring units (PMUs). Perf estimates energy use through the Running Average Power Limit (RAPL) interface, which the CPU periodically samples integrated energy counters. With CryptOracle, we query the RAPL Package domain, which captures the cumulative power drawn by the entire processor socket (cores, caches, and attached memory).

Runtime metrics are collected in this same pass because their acquisition introduces negligible overhead compared with event profiling or call-stack tracing. For each primitive operation, the profiler repeats the measurement several times and reports the median to reduce variance, a practice that is especially important for low-latency primitives.

Runtime analysis requires two passes. The first "regular" pass measures the full workload (e.g., matrix multiplication plus ciphertext setup through deserialization). The second

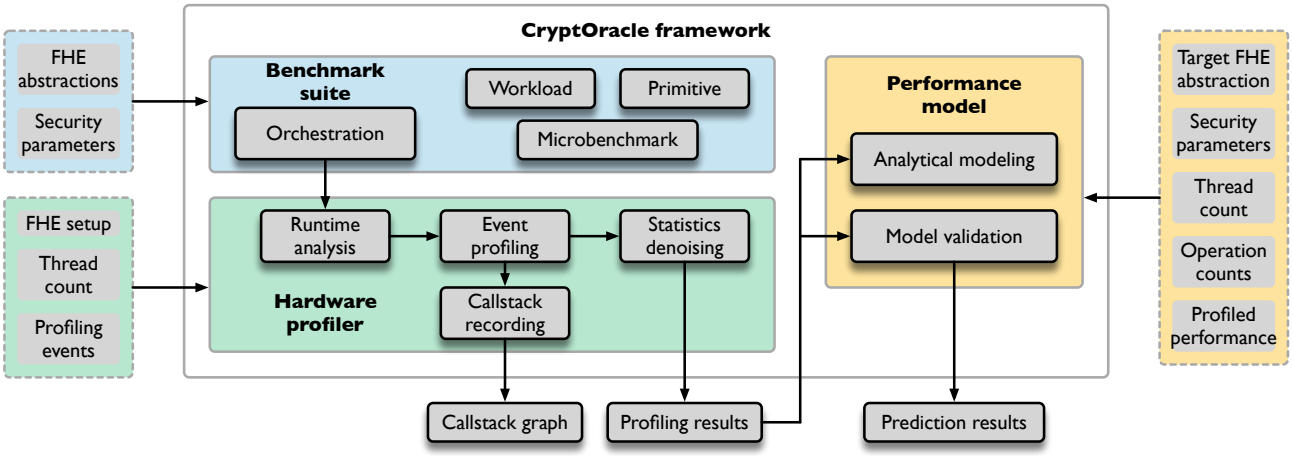

Fig. 3: Overview of CryptOracle framework.

"setup" pass isolates the initialization cost, measuring only the runtime and energy consumption of the setup steps (i.e., the initial deserialization of ciphertexts).

2) Event Profiling: The second phase of the hardware profiler carries out detailed event profiling with Linux Perf. Perf samples hardware registers at a specified frequency to count microarchitectural events. Users can choose any PMU events supported by the host CPU. Common selections include retired instructions, branches, branch mispredictions, and total cache references, as summarized in Table III.

For each unique benchmark configuration supplied by the orchestrator, the profiler executes the workload N times, where N is the user defined run count. The thread count is recorded as part of every event-profiling run.

TABLE III: Performance events currently collected.

Category	Event
Core	instructions, cpu-cycles, branches, branch-misses
Cache	cache-references, cache-misses, L1-dcache-loads, L1-icache-load-misses
TLB	dTLB-loads, dTLB-load-misses, iTLB-load-misses
Page Faults	page-faults, minor-faults

3) Statistics Denoising: After profiling, the raw statistics recorded during the setup phase are subtracted from those collected during the steady-state execution phase for each timing, energy, and perf counter, isolating the program's region of interest (ROI). For low-latency primitives, these values are averaged across all internal calls to ensure a sufficiently large sample. Next, average power consumption is calculated. All metrics are stored for subsequent data analysis and performance model construction.

4) Callstack Recording: During a third pass of the hardware profiler, users can record the application's call stack. Capturing this information reveals which FHE operations dominate execution time. The resulting stack trace is automatically

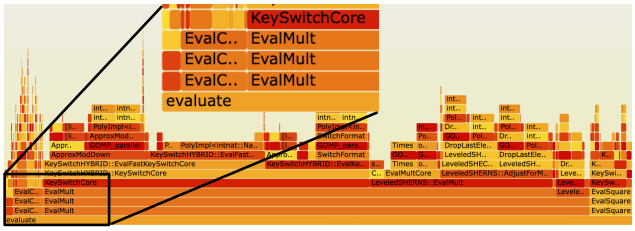

Fig. 4: FlameGraph of the logistic function.

rendered as a FlameGraph [27]. In a FlameGraph, the width of each box indicates the proportion of total runtime spent in the corresponding function, while the vertical axis represents call-stack depth. Figure 4 presents the logistic function, showing that roughly 90% of its runtime is consumed by the homomorphic multiply operation. This visualization enables developers to identify performance bottlenecks at early stages.

D. Performance Model

Packaged with the hardware profiler is a lightweight performance model that rapidly evaluates application runtime and other metrics for security parameters and thread counts that have not yet been tested. We use *application* to denote both microbenchmarks and full workloads. The model receives two inputs: the primitive profiling results and the counts of every homomorphic operation in the target application. It then linearly extrapolates the application runtime (e.g. ResNet-20 inference) by summing, for each primitive, the product of its count and its measured runtime at the chosen security parameters and thread count. Because FHE abstractions exhibit long runtimes, this linear approach provides projections with minimal overhead. This model could then be easily used to evaluate the impact of algorithmic changes to primitive operations or scaling an application to a higher security level. To assess the model's fidelity, CryptOracle includes a routine that validates the estimated runtime and reports the estimation error.

TABLE IV: Hardware specifications.

Specification	Intel i9-13900K	AMD Ryzen 9 7950X
Core/Thread Count	24/32	16/32
CPU Freq (GHz)	5.8	5.88
L1d/L1i Cache (KB)	896/1331	512/512
L2/L3 Cache (MB)	32/36	16/64
Total DRAM (GB)	64	96

IV. Experimental Setup

A. Software Setup

To understand the trade-offs between computational efficiency and security, we sweep the ciphertext ring dimension N across $[2^{13}, 2^{17}]$ and the computational depth L across [10, 20], maintaining a constant batch size B. We also profile the predefined security standards (128, 192, and 256-bit security from [5]), which override the default parameter selection. To ensure a fair comparison, security parameters, evaluation keys, and input ciphertexts are generated once, serialized to files, and then reused by all primitives and microbenchmarks. Each workload fixes any additional application-specific parameters. Table II lists the default settings used in the evaluation, unless stated otherwise. During data collection, the median of five runs is reported for each metric. For primitive profiling, where an individual operation may complete in tens of microseconds, we invoke each primitive repeatedly until the cumulative runtime exceeds 500ms, then compute per-call averages based on the total number of invocations.

B. Hardware Setup

Profiling was conducted on two CPU platforms, an Intel i9 and an AMD Ryzen 9 (see Table IV), to identify hardware-level performance differences between them. In OpenFHE, primitives and lower-level operations are parallelized with OpenMP. To study performance at each abstraction layer, we varied the number of OpenMP threads by setting the environment variable `OMP_NUM_THREADS`. Our evaluation targets fine-grained multithreading, where threads operate within a single primitive. In contrast, coarse-grained multithreading, in which threads are distributed across primitives, is disabled for both the microbenchmark and workload abstraction levels.

V. Evaluation

A. Profiling Results

1) Primitive Analysis: Figure 5 presents runtime and energy consumption for homomorphic encryption primitives on AMD and Intel CPUs under varying thread counts. On both platforms, the runtime of *EvalMult* and *EvalRotate* falls as the number of threads rises, but the benefit plateaus at 8 threads, suggesting limited scalability and a potential memory bottleneck. Figure 6(c) supports this interpretation, showing elevated cache reference counts on AMD CPUs beyond 8 threads. Because *EvalAdd* and *EvalAdd (Ptxt)* already run in about ∼2ms, they leave little room for further speed-up, and

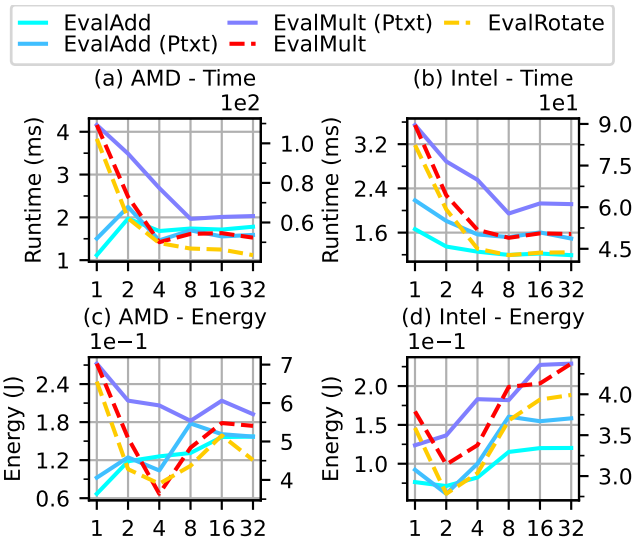

Fig. 5: Runtime and energy consumption of each primitive across the evaluated CPU vendors. The horizontal axis shows the thread counts, and the vertical axis is split into a left axis (cold-colored solid lines) and a right axis (warm-colored dashed lines). Together, these curves summarize performance across all tested platforms.

the OpenMP overhead incurred when moving from one to two threads can even cause a slight slowdown.

Energy consumption follows the same pattern as runtime across all kernels. Lightweight kernels maintain nearly constant energy use across thread counts, whereas compute-intensive primitives such as *EvalMult* and *EvalRotate* show an initial drop in energy with additional threads due to shorter runtimes, followed by an increase beyond 4 threads as parallelization overheads emerge. The results generated with CryptOracle emphasize its capacity to capture precise performance metrics across diverse hardware platforms.

Figure 6 explores the microarchitectural behaviour of the primitives on the AMD platform, presenting a subset of the events listed in Table III. At low thread counts, *EvalMult* and *EvalRotate* record the largest instruction footprints, underscoring their computational intensity. As the thread count rises from one to four, the instruction counts drop substantially, most notably for *EvalMult*, indicating that multithreading removes redundant serial work. In contrast, cache references increase with additional threads because of greater data sharing. The pattern of dTLB loads follows the cache trend, as increased parallel memory accesses raise TLB pressure and introduce page-level contention. Correspondingly, page-fault rates fall as more threads are added. Lightweight primitives such as *EvalAdd* show only modest changes across all counters. As homomorphic encryption kernels continue to evolve, CryptOracle will offer detailed insights into their architectural challenges.

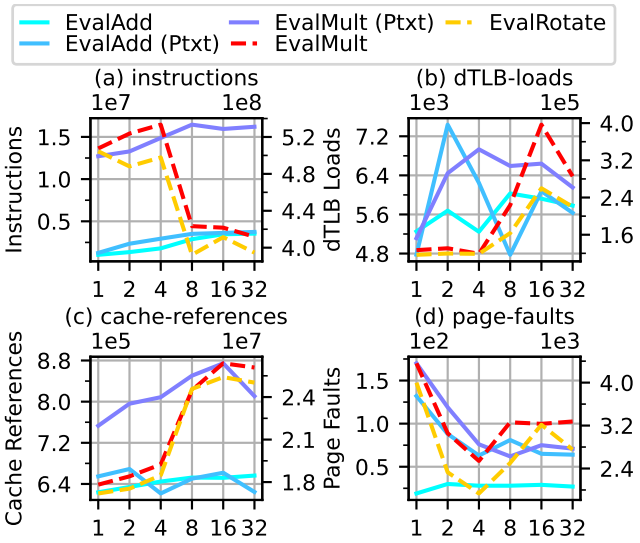

Fig. 6: Primitive architecture statistics on an AMD CPU plotted against thread count on the x axis.

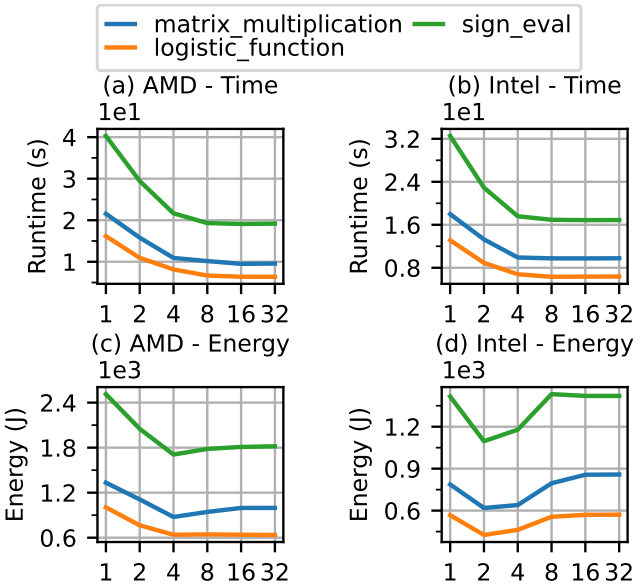

Fig. 7: Runtime and energy microbenchmarks across CPU vendors with thread count plotted on the x axis.

2) OpenFHE Hardware Backends: While this work explores profiling OpenFHE on CPUs, OpenFHE is designed as a modular library able to interface with GPUs and accelerators via a hardware backend, if available. Currently, the only officially-supported OpenFHE backend is the Homomorphic Encryption Accelerator Library (HEXL). This library accelerates integer arithmetic on devices supporting AVX512 instructions. Figure 8 compares an EvalMult operation's runtime and energy consumption with and without HEXL acceleration, which both exhibit similar trends for increasing

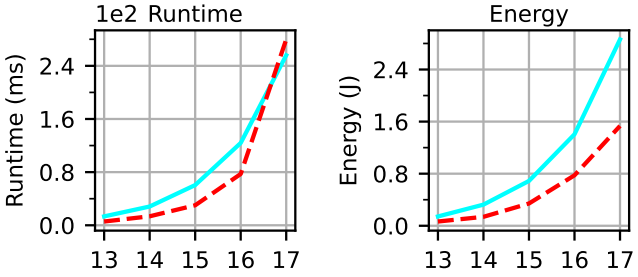

Fig. 8: Single-threaded runtime and energy consumption of baseline (solid) and HEXL (dashed) backends for the EvalMult primitive across ring dimensions on AMD CPU.

ring dimension. HEXL provides an average speedup of 1.8x, with a maximum speedup of 2.36x at $N = 2^{16}$, after which memory becomes the bottleneck in the operation at $N = 2^{17}$.

3) Microbenchmark Analysis: We further analyze three microbenchmarks and several matrix sizes for matrix multiplication. A similar profile could be generated for the full workloads, but we omit it for brevity.

Figure 7 presents both runtime and energy for the three microbenchmarks. As expected, increasing the number of CPU threads reduces runtime on both AMD and Intel processors. The benefit, however, diminishes quickly: the runtime curve flattens at about 8 threads on each platform.

Energy consumption follows a different pattern. The minimum energy appears at 4 threads on the AMD system and at 2 threads on the Intel system for almost every microbenchmark. Beyond these points, the small runtime savings cannot offset the additional energy required by extra threads. On the Intel CPU, using 8 to 32 threads even consumes more energy than a single thread. One likely reason is the heterogeneous design of the Intel i9-13900K, which combines high-performance (P) cores with energy-efficient (E) cores and complicates thread scheduling. These observations reveal a clear performance-efficiency trade-off that can guide the design of more sustainable applications.

Figure 9 depicts the runtime scaling of matrix multiplication. When scaling up the ring dimension (from 2^{14} to 2^{17}), the runtime grows almost linearly. Holding the ring dimension fixed and enlarging the matrix size also yields a linear rise, for instance an approximately 2.25× slowdown when the ring dimension is 17 with one thread (the highest point on the blue curve). This linear behavior under FHE contrasts with the quadratic scaling characteristic of plaintext matrix multiplication. Finally, as the thread count increases, larger matrices reach saturation sooner, causing the corresponding curves to converge.

4) Profiling Overhead: Table V summarizes the profiling overhead incurred by CryptOracle. The program region of interest encompasses only the FHE computation itself. The runtime analysis overhead corresponds to the initialization cost required to deserialize ciphertexts. This cost is relatively low and vendor independent, about 0.05s on both AMD and

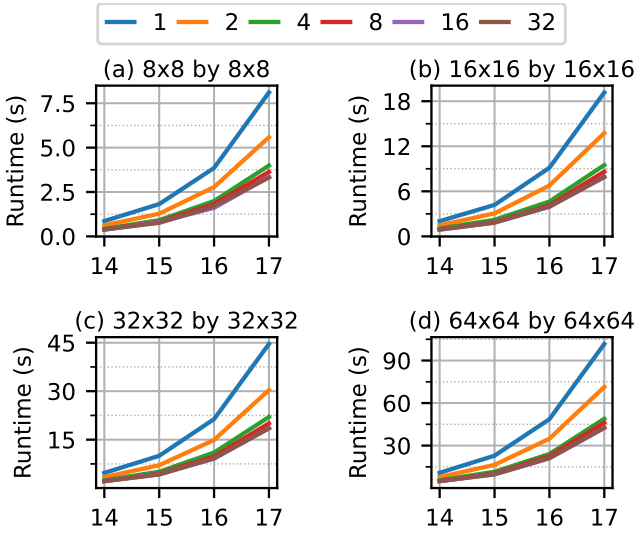

Fig. 9: Matrix Mult runtime measured on an AMD CPU, with separate panels for matrix sizes, the x axis for ring dimension (powers of 2), and color coding for thread counts.

TABLE V: Profiling overhead of microbenchmarks over ROI for 8 threads, N=16, and depth=10. '-A' and '-I' denote AMD and Intel CPUs. $\Delta\%$ denotes the relative overhead.

Name	Time (s)		
	ROI	Runtime analysis ($\Delta\%$)	Event profiling ($\Delta\%$)
Logistic Func-A	6.68	0.04 (0.60%)	4.06 (60.8%)
Matrix Mult-A	10.16	0.04 (0.39%)	5.78 (56.9%)
Sign Eval-A	19.20	0.05 (0.26%)	11.83 (61.6%)
Logistic Func-I	6.24	0.05 (0.80%)	3.39 (54.3%)
Matrix Mult-I	9.77	0.05 (0.51%)	2.75 (28.1%)
Sign Eval-I	16.78	0.05 (0.30%)	5.43 (32.4%)

Intel CPUs. The event-profiling overhead reflects the extra time needed to gather performance counters. This overhead is substantially larger than that of runtime analysis and varies with both the microbenchmark and the vendor. AMD displays relatively stable overheads of roughly 60%, whereas Intel exhibits greater fluctuation but a lower relative overhead.

B. Performance Model

1) Runtime and Energy Prediction: There are two observations in Figure 10. First, in general, the model exhibits better prediction accuracy compared to workloads, since microbenchmarks have been well optimized for FHE challenges [23], while workloads are more for functional evaluations. Second, larger thread counts give better accuracy, as the performance saturates more. The geomean of prediction accuracy in dashed lines concurs with the two observations above. The overall geomean across all thread counts is -7.02% and 8.40% for microbenchmark and workload runtime, and -9.74% and 15.67% for microbenchmark and workload energy.

2) Prediction Breakdown: Figure 11 shows the contribution of primitives, expressed as a percentage of runtime or energy.

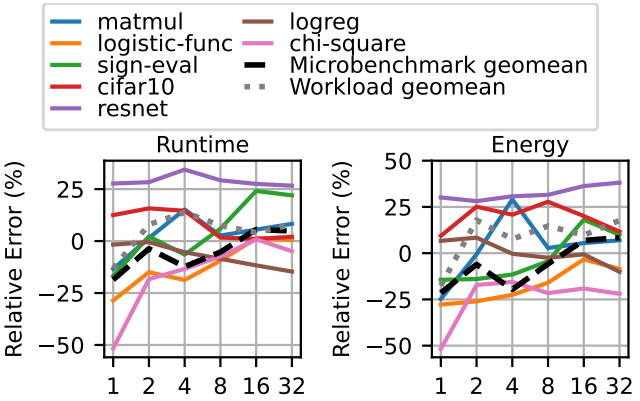

Fig. 10: Prediction analysis of relative error across threads on AMD CPU.

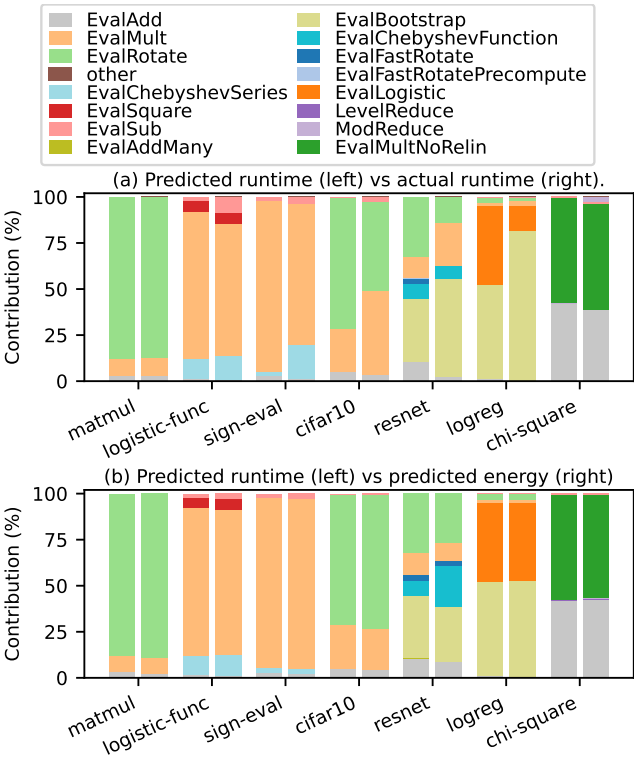

Fig. 11: Primitive-level contributions for runtime and energy on an AMD CPU using 8 threads, shown across microbenchmarks and workloads (x-axis).

Figure 11 (a) shows the predicted vs actual runtime breakdown for various workloads, while Figure 11 (b) contrasts predicted runtime vs predicted energy contributions. Each bar is stacked by the type of primitive kernels used in the application. Though we show a thorough breakdown, there is scope for "other" newer kernels to be supported by CryptOracle for future workloads. Figure 11 (a) indicates strong alignment between predicted and actual runtime across microbenchmarks.

TABLE VI: Prediction overhead of workloads for 8 threads. '-A' and '-I' denote AMD and Intel CPUs. The profiling time here is for runtime analysis.

Name	Time (s)		Speedup
	Profiling	Prediction	
Chi Square Test-A	49.3	0.6	82.2×
CIFAR-10-A	2.05		3.4×
Logistic Regression-A	253.2		422.0×
ResNet-20-A	236.8		394.7×
Chi Square Test-I	35.5		59.2×
CIFAR-10-I	1.85		3.1×
Logistic Regression-I	231.5		385.8×
ResNet-20-I	217.5		362.5×

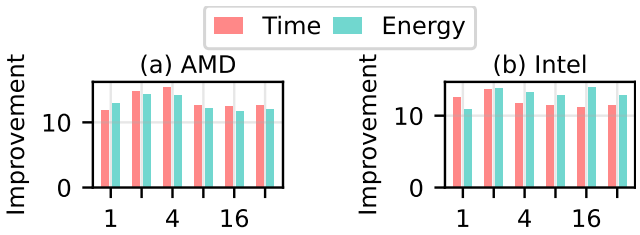

Fig. 12: Estimated improvements of runtime and energy across different CPU vendors and thread counts (x axis). The algorithm configurations are d=128, r=2, N=16, depth=3 for "Algorithm 5", and depth=6 for "Rectangular MM" [48].

While this deviates more for workloads (e.g. cifar10, resnet and logreg) due to their more complex nature, the prediction preserves the relative importance of primitives. Figure 11 (b) highlights the difference between predicted runtime and predicted energy, with most applications showing a similar distribution for each. Resnet exhibits a slight deviation, with EvalChebyshevFunction occupying a higher energy ratio than runtime. Thus, our results indicate that CryptOracle accurately preserves the relative contribution of individual primitives, capturing consistent trends in both runtime and energy profiles.

3) Prediction Overhead: We present the prediction overhead of CryptOracle in Table VI. The table compares the runtime analysis time, which measures execution time and energy without collecting performance counters, against the model prediction time, which is an almost constant cost incurred by running the Python-based performance model. Profiling latency varies considerably across workloads, and Intel CPUs consistently outperform AMD CPUs because of their larger on-chip memory capacities, in agreement with the primitive and microbenchmark results. The performance model accelerates runtime and energy analysis by $10\times \sim 500\times$, depending on the workload.

4) Case Study: Algorithmic Innovation: To illustrate another application of CryptOracle, we estimate the runtime and energy savings that can be achieved through algorithmic innovations. Specifically, we count the primitive operations required by two algorithms implementing ciphertext–ciphertext matrix multiplication [48]. The projected gains are presented in Figure 12. Across various CPU vendors and thread counts, the projected speedups and energy reductions are consistent, ranging from $10\times \sim 15\times$. The original study in [48] evaluated its implementation on a GPU; the measured speedup there is approximately 5.42×, which differs from our CPU-based projection.

VI. Limitations of This Work

We identify two primary limitations in this work. First, CryptOracle currently supports only CPU platforms due to the lack of supported OpenFHE hardware backends. This limitation does not diminish the merit of CryptOracle, as it remains an accessible platform for examining advances in FHE applications and algorithms. Also, GPU acceleration of FHE has shown great promise, and CryptOracle's infrastructure can be expanded to support future GPU extensions to OpenFHE. Some of these extensions already exist [3], but they are experimental and lack any full applications to benchmark. Second, CryptOracle does not yet predict performance or efficiency on unseen hardware or algorithm configurations. This gap could be addressed by machine learning-based techniques instead of simple linear extrapolation. Similar strategies have been extensively studied for AI workloads [29], [38], [47], and we leave their adoption to future work.

VII. Related Work

Although traditional benchmark suites such as SPEC CPU2017 [10], MLPerf [41], and Graph500 [26] have been extensively studied, systematic performance evaluation of FHE remains sparse, but on the horizon.

Pal et al. [44] present a foundational study using OpenFHE [6], SEAL [51], and HEaaN [14], revealing runtime and energy trade-offs across security levels, polynomial degrees, and threading models. FHEBench [30] supplies a standardized suite for latency and memory comparison across libraries, and Bergerat et al. [7] explore parameter optimization for TFHE.

HEBench [24], created by Intel and collaborators, evaluates canonical workloads such as dot products and matrix multiplication through a modular backend loader. Despite its robustness, it currently supports only a small number of libraries. HEProfiler [53] offers cycle accurate profiling of FHE primitives, exposing threading and memory inefficiencies. The native benchmark suite in OpenFHE [6] provides only basic profiling, limited to runtime, CPU core, and frequency statistics. It omits advanced performance metrics such as latency, power, and throughput and lacks event profiling, runtime tracing, and other profiling capabilities.

These studies supply valuable empirical evidence but remain focused on microbenchmarking and parameter tuning. In contrast, CryptOracle profiles both primitives and end-to-end workloads while also delivering accurate, predictive models that guide system-level optimization.

VIII. Conclusion

As privacy-preserving machine learning becomes increasingly critical, FHE offers strong theoretical security guarantees but also incurs considerable computational overhead, restricting practical deployment. We introduced CryptOracle, a modular characterization framework that integrates a benchmark suite, a hardware profiler, and a performance model to understand the runtime and energy consumption of CPUs running OpenFHE. Our open-source release of CryptOracle aims to accelerate progress toward making FHE practical for real-world machine learning workloads.

Acknowledgment

This research is in part sponsored by the AMD University Program, in the form of machine donation and student fellowships to the University of Central Florida.

References

[1] "Lattigo v6," Online: https://github.com/tuneinsight/lattigo, Aug. 2024, ePFL-LDS, Tune Insight SA.

[2] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 882–895.

[3] C. Agulló-Domingo, Óscar Vera-López, S. Guzelhan, L. Daksha, A. E. Jerari, K. Shivdikar, R. Agrawal, D. Kaeli, A. Joshi, and J. L. Abellán, "Fideslib: A fully-fledged open-source fhe library for efficient ckks on gpus," 2025. [Online]. Available: https://arxiv.org/abs/2507.04775

[4] E. Aharoni, A. Adir, M. Baruch, N. Drucker, G. Ezov, A. Farkash, L. Greenberg, R. Masalha, G. Moshkowich, D. Murik *et al.*, "Helayers: A tile tensors framework for large neural networks on encrypted data," *arXiv preprint arXiv:2011.01805*, 2020.

[5] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, S. Lokam, D. Micciancio, D. Moody, T. Morrison, A. Sahai, and V. Vaikuntanathan, "Homomorphic encryption security standard," HomomorphicEncryption.org, Toronto, Canada, Tech. Rep., November 2018.

[6] A. A. Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, "OpenFHE: Open-source fully homomorphic encryption library," Cryptology ePrint Archive, Paper 2022/915, 2022, https://eprint.iacr.org/2022/915. [Online]. Available: https://eprint.iacr.org/2022/915

[7] L. Bergerat, A. Boudi, Q. Bourgerie, I. Chillotti, D. Ligier, J.-B. Orfila, and S. Tap, "Parameter optimization and larger precision for (t) fhe," *Journal of Cryptology*, vol. 36, no. 3, p. 28, 2023.

[8] W. L. Bircher and L. K. John, "Complete system power estimation using processor performance events," *IEEE Transactions on Computers*, vol. 61, no. 4, pp. 563–577, 2011.

[9] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *ACM Transactions on Computation Theory (TOCT)*, vol. 6, no. 3, pp. 1–36, 2014.

[10] J. Bucek, K.-D. Lange, and J. v. Kistowski, "Spec cpu2017: Next-generation compute benchmark," ser. ICPE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 41–42. [Online]. Available: https://doi.org/10.1145/3185768.3185771

[11] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, p. 991–1008. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/bulck

[12] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part I 37*. Springer, 2018, pp. 360–384.

[13] ——, "A full rns variant of approximate homomorphic encryption," in *International Conference on Selected Areas in Cryptography*. Springer, 2018, pp. 347–368.

[14] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in cryptology–ASIACRYPT 2017: 23rd international conference on the theory and applications of cryptology and information security, Hong kong, China, December 3-7, 2017, proceedings, part i 23*. Springer, 2017, pp. 409–437.

[15] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Tfhe: fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.

[16] A. Crapé and L. Eeckhout, "A rigorous benchmarking and performance analysis methodology for python workloads," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2020, pp. 83–93.

[17] J. Demme, R. Martin, A. Waksman, and S. Sethumadhavan, "Side-channel vulnerability factor: A metric for measuring information leakage," in *2012 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 106–117.

[18] L. Ducas and D. Micciancio, "Fhew: bootstrapping homomorphic encryption in less than a second," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015, pp. 617–640.

[19] Z. Durumeric, F. Li, J. Kasten, J. Amann, J. Beekman, M. Payer, N. Weaver, D. Adrian, V. Paxson, M. Bailey, and J. A. Halderman, "The matter of heartbleed," in *Proceedings of the 2014 Conference on Internet Measurement Conference*, ser. IMC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 475–488. [Online]. Available: https://doi.org/10.1145/2663716.2663755

[20] A. Ebel, K. Garimella, and B. Reagen, "Orion: A fully homomorphic encryption framework for deep learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 734–749.

[21] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptology ePrint Archive*, 2012.

[22] S. Fan, Z. Wang, W. Xu, R. Hou, D. Meng, and M. Zhang, "Tensorfhe: Achieving practical computation on encrypted data using gpgpu," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 922–934.

[23] Fherma-challenges contributors, "Fherma-challenges," https://github.com/Fherma-challenges, 2025, accessed: 2025-06-05.

[24] A. Foundation, CryptoLab, Deloitte, Duality, I. Research, Intel, K. Leuven, M. Research, and T. I. SA, "Homomorphic encryption benchmarking framework - hebench (release v0.9)," Online: https://github.com/hebench/frontend, Nov. 2022.

[25] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.

[26] Graph500 Committee, "Graph500 benchmark," https://graph500.org/, 2024, accessed: 2025-05-27.

[27] B. Gregg, "Flamegraph: Stack trace visualizer," https://github.com/brendangregg/FlameGraph, 2017.

[28] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the gan: Information leakage from collaborative deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 603–618. [Online]. Available: https://doi.org/10.1145/3133956.3134012

[29] M. Isaev, N. Mcdonald, L. Dennison, and R. Vuduc, "Calculon: A Methodology and Tool for High-Level Co-Design of Systems and Large Language Models," in *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

[30] L. Jiang and L. Ju, "Fhebench: Benchmarking fully homomorphic encryption schemes," *arXiv preprint arXiv:2203.00728*, 2022.

[31] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, "Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 114–148, 2021.

[32] W. Jung, E. Lee, S. Kim, J. Kim, N. Kim, K. Lee, C. Min, J. H. Cheon, and J. H. Ahn, "Accelerating fully homomorphic encryption through architecture-centric analysis and optimization," *IEEE Access*, vol. 9, pp. 98 772–98 789, 2021.

[33] J. Kim, G. Lee, S. Kim, G. Sohn, M. Rhu, J. Kim, and J. H. Ahn, "Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1237–1254.

[34] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. H. Ahn, "Bts: An accelerator for bootstrappable fully homomorphic encryption," in *Proceedings of the 49th annual international symposium on computer architecture*, 2022, pp. 711–725.

[35] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1–19.

[36] K. E. Lauter, W. Dai, and K. Laine, *Protecting privacy through homomorphic encryption*. Springer, 2022.

[37] J. Lee, E. Lee, J.-W. Lee, Y. Kim, Y.-S. Kim, and J.-S. No, "Precise approximation of convolutional neural networks for homomorphically encrypted data," *IEEE Access*, vol. 11, pp. 62 062–62 076, 2023.

[38] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting gpu performance for deep learning training and inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 493–508. [Online]. Available: https://doi.org/10.1145/3669940.3707265

[39] C. Liu, M. Hicks, and E. Shi, "Memory trace oblivious program execution," in *2013 IEEE 26th Computer Security Foundations Symposium*, 2013, pp. 51–65.

[40] F. Liu, N. R. Miniskar, D. Chakraborty, and J. S. Vetter, "Deffe: a data-efficient framework for performance characterization in domain-specific computing," in *Proceedings of the 17th ACM International Conference on Computing Frontiers*, 2020, pp. 182–191.

[41] P. Mattson, C. Cheng, C. Coleman, G. Diamos, P. Micikevicius, D. Patterson, H. Tang, G.-Y. Wei, P. Bailis, V. Bittorf, D. Brooks, D. Chen, D. Dutta, U. Gupta, K. Hazelwood, A. Hock, X. Huang, A. Ike, B. Jia, D. Kang, D. Kanter, N. Kumar, J. Liao, G. Ma, D. Narayanan, T. Oguntebi, G. Pekhimenko, L. Pentecost, V. J. Reddi, T. Robie, T. S. John, T. Tabaru, C.-J. Wu, L. Xu, M. Yamazaki, C. Young, and M. Zaharia, "Mlperf training benchmark," 2020. [Online]. Available: https://arxiv.org/abs/1910.01500

[42] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: the case of AES," Cryptology ePrint Archive, Paper 2005/271, 2005. [Online]. Available: https://eprint.iacr.org/2005/271

[43] A. Pal, K. Desai, R. Chatterjee, and J. San Miguel, "Camouflage: Utility-aware obfuscation for accurate simulation of sensitive program traces," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, May 2024. [Online]. Available: https://doi.org/10.1145/3650110

[44] S. Pal, K. Swaminathan, E. Aharoni, E. Kushnir, N. Drucker, H. Schaul, A. Buyuktosunoglu, O. Soceanu, and P. Bose, "Fully homomorphic encryption for computer architects: A fundamental characterization study," in *Annual IEEE/ACM International Symposium on Microarchitecture*, 2023.

[45] Y. Park, A. Amarnath, S. Pal, K. Swaminathan, A. Buyuktosunoglu, H. Shaul, E. Aharoni, N. Drucker, W. D. Lu, O. Soceanu *et al.*, "Fhendi: A near-dram accelerator for compiler-generated fully homomorphic encryption applications," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1127–1142.

[46] Y. Polyakov, K. Rohloff, G. W. Ryan, and D. Cousins, "Palisade lattice cryptography library user manual," *Cybersecurity Research Center, New Jersey Institute ofTechnology (NJIT), Tech. Rep*, vol. 15, 2017.

[47] H. Qi, E. R. Sparks, and A. Talwalkar, "Paleo: A Performance Model for Deep Neural Networks," in *International Conference on Learning Representations*, 2017.

[48] D. Rho, T. Kim, M. Park, J. W. Kim, H. Chae, E. K. Ryu, and J. H. Cheon, "Encryption-friendly llm architecture," *arXiv preprint arXiv:2410.02486*, 2024.

[49] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, "F1: A fast and programmable accelerator for fully homomorphic encryption," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 238–252.

[50] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, "Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 173–187.

[51] "Microsoft SEAL (release 4.1)," https://github.com/Microsoft/SEAL, Jan. 2023, microsoft Research, Redmond, WA.

[52] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 3–18.

[53] J. Takeshita, N. Koirala, C. McKechney, and T. Jung, "Heprofiler: An in-depth profiler of approximate homomorphic encryption libraries," *Journal of Cryptographic Engineering*, vol. 15, no. 2, pp. 1–26, 2025.

Appendix

A. Abstract

The artifact appendix describes how to access the artifacts of the CryptOracle framework introduced in Figure 3. It also describes how to regenerate included plots for the paper and how to reproduce representative results for Figures 4, 5, 7, 10 and 11.

B. Artifact check-list (meta-information)

- **Compilation**: GCC
- **Run-time environment**: C++, Python3
- **Hardware**: Linux-based CPUs, AMD and Intel
- **Metrics**: Latency, energy consumption, hardware events (via perf)
- **Output**: Profiling CSV reports, FlameGraph SVGs, and figure plots
- **Experiments**: Figures 4, 5, 7, 10 and 11
- **How much disk space required (approximately)?**: < 10 GB
- **How much time is needed to prepare workflow (approximately)?**: Expect ~30 minutes.
- **How much time is needed to complete experiments (approximately)?**: Expect ~2 hours.
- **Publicly available?**: Yes
- **Code licenses (if publicly available)?**: MIT License (LICENSE file in the GitHub repository)

C. Description

1) How to access: The CryptOracle artifact is available on Zenodo (DOI: 10.5281/zenodo.18904807). It is also available on GitHub: https://github.com/UnaryLab/CryptOracle.

2) Hardware dependencies:

- AMD or Intel CPU
- 16GB of RAM or more
- ~10GB of disk space

3) Software dependencies:

- Linux OS. Tested on Ubuntu 22.04. WSL is unsupported and was not validated (missing full support for perf).
- Libraries: all required libraries are listed in Installation section or handled by setup scripts.

D. Installation

For artifact evaluation, please follow these steps:

1. Clone the GitHub repository and checkout AE branch

```
git clone https://github.com/UnaryLab/CryptOracle
cd CryptOracle
git checkout ispass-ae
```

2. Install required dependencies

```
sudo apt-get update -y
sudo apt install -y \
  git cmake autoconf \
  build-essential libtool \
  libgoogle-perftools-dev python3-dev \
  python3-pip python3-venv \
  libboost-all-dev "linux-tools-$(uname -r)" \
  linux-tools-common linux-tools-generic
```

Note: "perf" may not exist for your specific kernel; run the following to use the latest kernel implementation available (stability not guaranteed):

```
sudo ln -sf "$(ls -d /usr/lib/linux-tools-* | sort -V
     | tail -n1)/perf" /usr/local/bin/perf
```

3. Enable hardware performance counters

```
# Temporarily enable counters
echo -1 | sudo tee /proc/sys/kernel/
    perf_event_paranoid

# Persist across reboots
sudo sed -i '/^kernel\.perf_event_paranoid/d'
    /etc/sysctl.conf
echo "kernel.perf_event_paranoid = -1" | sudo tee -a
    /etc/sysctl.conf

# Allow unrestricted kernel symbol access
sudo sed -i '/^kernel\.kptr_restrict/d'
    /etc/sysctl.conf
echo "kernel.kptr_restrict = 0" | sudo tee -a
    /etc/sysctl.conf
sudo sysctl -p
```

CryptOracle's profiler requires elevated kernel permissions to access hardware performance counters. If reviewers do not wish to grant these permissions, they may skip subsections F2 and F3.

4. Create a Python virtual environment

```
python3 -m venv venv
source ./venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

5. Clone and setup the primitive, microbenchmark, and workload directories

```
python3 benchmark-main.py --setup-benchmarking
```

6. Add the OpenFHE install to your environment PATH

```
LIB_DIR="$(pwd)/openfhe-development-install/lib"
export LD_LIBRARY_PATH=\
"$LIB_DIR${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"
```

E. Experiment workflow

Now that the CryptOracle framework is set up, we will run the artifact to reproduce key results from the paper. It is not possible for us to provide direct access to the machines

used to produce our results, but we can provide instructions to collect new profiling data on x86 CPUs. Due to machine-specific variation, exact agreement in results is not expected, but generated figures should preserve the trends and relative comparisons reported in the paper.

F. Evaluation and expected results

1) Recreating plots from collected data (all figures):

```
python3 ae-scripts/generate-ispass-plots.py
```

This will generate all of the figures from the paper from the included profiling datasets under *ae-plots*/.

2) Collecting new primitive data (figure 5):

```
bash ae-scripts/collect_figure_5_data.sh
python3 ae-scripts/generate_figure_5_plot.py
```

This script will collect profiling data for a sweep of security and thread parameters, specified under *ae-config/parameters.yaml*. This script follows the standard methodology for collecting results in the paper: collecting five runs for each parameter combination, then recording the median run. CSV results are generated in *out/csv/primitive-results-ae.csv*. By default, the plotting script will use this path and save the plot to *ae-plots/figure_5_plot.pdf*.

3) Collecting new microbenchmark data (figure 7):

```
bash ae-scripts/collect_figure_7_data.sh
python3 ae-scripts/generate_figure_7_plot.py
```

This example collects runtime and energy metrics for the three microbenchmarks, which are shown in Figure 7. CSV results are generated in *out/csv/microbenchmark-results-ae.csv*. By default, the plotting script will use this path and save the plot to *ae-plots/figure_7_plot.pdf*.

4) Using performance model (figures 10 and 11):

```
bash ae-scripts/estimate_matmul_performance.sh
```

This example involves using the performance model to estimate the performance of matrix multiplication based on the operation counts. This reproduces a subset of the results in Figures 10 and 11, corresponding to matrix multiplication. The figures and CSV will be produced in the *ae-plots* directory as *matrix_multiplication_analysis.png*, *matrix_multiplication_operation_contributions.png*, and *estimated_matrix_multiplication_median.csv*.

5) Generating FlameGraphs (figure 4):

```
bash ae-scripts/generate_figure_4_flamegraph.sh
```

To demonstrate CryptOracle's ability to automatically generate FlameGraphs for any primitive, microbenchmark, or workload, this example produces a FlameGraph for the Logistic Function microbenchmark. A similar FlameGraph can be found in Figure 4 of the paper, which has been post-processed for readability. After running the command above, the FlameGraph will be written to *out/flamegraphs* as *ae_logistic_function_FlameGraph.svg*.

G. Experiment customization

To profile additional configurations, users can make modifications to the default yaml files in the *in/* directory. Additionally, they can create their own yaml config files and specify the path when calling *benchmark-main.py*. For the performance model, template config files are available under *perf-model* for estimating the performance of other supported microbenchmarks and workloads.